

P3107R3

Permit an efficient implementation of `std::print`

Published Proposal, 2024-03-18

Author:

[Victor Zverovich](#)

Audience:

LEWG

Project:

ISO/IEC 14882 Programming Languages — C++, ISO/IEC JTC1/SC22/WG21

Table of Contents

- 1 Introduction
- 2 Changes since R2
- 3 Changes since R1
- 4 Changes since R0
- 5 Problems
- 6 Proposal
- 7 Performance
- 8 Implementation
- 9 Wording
- 10 Acknowledgements

References

Informative References

§ 1. Introduction

C++23 introduced a new formatted output facility, `std::print` ([P2093]). It was defined in terms of formatting into a temporary `std::string` to simplify the specification and to clearly indicate the requirement for non-interleaved output. Unfortunately, it was discovered that this approach does not allow for a more efficient implementation strategy, such as writing directly to a stream buffer under a lock, as reported in [LWG4042]. This paper proposes a solution to address this shortcoming.

§ 2. Changes since R2

- Change the opt-in mechanism to use a namespace-scoped variable template instead of a nested member to prevent the inheritance issue.

§ 3. Changes since R1

- Made the new behavior an opt in for user-defined formatters to prevent potential deadlocks when they perform locking in their `format` functions.
- Added a missing `stream` argument in the call to `print` in the *Effects* clause of `println`.
- Added instructions to update the `__cpp_lib_print` feature testing macro.
- Provided an example illustrating a problem with interleaved output.
- Provided an example illustrating a problem with locking in C++ and Java.

§ 4. Changes since R0

- Added preliminary results for libstdc++ provided by Jonathan Wakely.
- Replaced the definition of `println` with a more efficient one that doesn't call `format`.
- Fixed typos.

§ 5. Problems

As reported in [LWG4042], `std::print/std::vprint*` is currently defined in terms of formatting into a temporary `std::string`, e.g. `[print.fun]`:

```
void vprint_nonunicode(FILE* stream, string_view fmt, format_args args);
```

Preconditions: `stream` is a valid pointer to an output C stream.

Effects: Writes the result of `vformat(fmt, args)` to `stream`.

Throws: Any exception thrown by the call to `vformat` (`[format.err.report]`). `system_error` if writing to `stream` fails. May throw `bad_alloc`.

This prohibits a more efficient implementation strategy of formatting directly into a stream buffer under a lock (`flockfile/funlockfile` in POSIX, `[STDIO-LOCK]`) like C `stdio` and other formatting facilities do.

The inability to achieve this with the current wording stems from the observable effects: throwing an exception from a user-defined formatter currently prevents any output from a formatting function, whereas with the direct method, the output written to the stream before the exception occurred is preserved. Most errors are caught at compile time, making this situation uncommon. The current behavior can be easily replicated by explicitly formatting into an intermediate string or buffer.

Another problem is that such double buffering may require unbounded memory allocations, making `std::print` unsuitable for resource-constrained applications creating incentives for continued use of unsafe APIs. In the direct method, there are usually no memory allocations.

§ 6. Proposal

The current paper proposes expressing the desire to have non-interleaved output in a way that permits a more efficient implementation similar to `printf`'s. It is based on the locking mechanism provided by C streams, quoting Section 7.21.2 Streams of the C standard (`[N2310-STREAMS]`):

7 Each stream has an associated lock that is used to prevent data races when multiple threads of execution access a stream, and to restrict the interleaving of stream operations performed by multiple threads. Only one thread may hold this lock at a time. The lock is reentrant: a single thread may hold the lock multiple times at a given time.

8 All functions that read, write, position, or query the position of a stream lock the stream before accessing it. They release the lock associated with the stream when the access is complete.

As shown in [Performance](#), this can give more than 20% speed up even compared to writing to a stack-allocated buffer.

All of the following languages use an implementation consistent with the current proposal (no intermediate buffering):

- C (`printf`)
- Rust (`println!`)
- Java (`System.out.format`)

IOStreams don't provide atomicity which is even weaker than the guarantees provided by these languages and the current proposal. For example:

```
#include <iostream>
#include <thread>

void worker() {
    for (int i = 0; i < 3; ++i) {
        // Simulate work.
        std::this_thread::sleep_for(std::chrono::milliseconds(200));
        std::cout << "thread " << std::this_thread::get_id()
                  << ": work" << i << " done\n";
    }
}

int main() {
    auto t1 = std::jthread(worker);
    auto t2 = std::jthread(worker);
}
```

may produce the following output:

```
thread 140239754491456: work0 done
thread 140239746098752: work0 done
thread thread 140239746098752: work140239754491456: work1 done
1 done
thread 140239754491456: work2 done
thread 140239746098752: work2 done
```

Neither `printf` nor `std::print` have this issue.

One problem with locking a stream is that it may introduce potential for deadlocks in case a user-defined formatter is also doing locking internally. For example:

```
struct deadlockable {
    int value = 0;
    mutable std::mutex mutex;
};

template <> struct std::formatter<deadlockable> {
    constexpr auto parse(std::format_parse_context& ctx) {
        return ctx.begin();
    }

    auto format(const deadlockable& d, std::format_context& ctx) const {
        std::lock_guard<std::mutex> lock(d.mutex);
        return std::format_to(ctx.out(), "{}", d.value);
    }
};

deadlockable d;
auto t = std::thread( [&]() {
    std::print("start\n");
    std::lock_guard<std::mutex> lock(d.mutex);
    for (int i = 0; i < 1000000; ++i) d.value += 10;
    std::print("done\n");
});
for (int i = 0; i < 100; ++i) std::print("{}", d);
t.join();
```

The same problem exists in other languages, for example:

```
class Deadlockable {
    public int value;
    public String toString() {
        synchronized (this) {
            return Integer.toString(value);
        }
    }
}

class Hello {
    public static void main(String[] args) throws InterruptedException {
        Deadlockable d = new Deadlockable();

        Thread t = new Thread(new Runnable() {
            private Deadlockable d;

            public Runnable init(Deadlockable d) {
                this.d = d;
                return this;
            }

            @Override
            public void run() {
                System.out.println("start");
                synchronized (d) {
                    for (int i = 0; i < 1000000; ++i) d.value += 10;
                    System.out.format("done");
                }
            }
        }).init(d));
        t.start();
        for (int i = 0; i < 100; ++i) System.out.format("%s", d);
        t.join();
    }
}
```

Found one Java-level deadlock:

=====

"main":

waiting to lock monitor 0x0000600002fb4750 (object 0x000000070fe120e8, a

which is held by "Thread-0"

"Thread-0":

waiting for ownable synchronizer 0x000000070fe08998, (a java.util.concurrent.locks.ReentrantLock\$Sync which is held by "main")

Java stack information for the threads listed above:

=====

"main":

at Deadlockable.toString(Hello.java:5)
- waiting to lock <0x000000070fe120e8> (a Deadlockable)
at java.util.Formatter\$FormatSpecifier.printString(java.base@21.0.2/Formatter.java:2791)
at java.util.Formatter\$FormatSpecifier.print(java.base@21.0.2/Formatter.java:2791)
at java.util.Formatter.format(java.base@21.0.2/Formatter.java:2791)
at java.io.PrintStream.implFormat(java.base@21.0.2/PrintStream.java:1367)
at java.io.PrintStream.format(java.base@21.0.2/PrintStream.java:1346)
at Hello.main(Hello.java:32)

"Thread-0":

at jdk.internal.misc.Unsafe.park(java.base@21.0.2/Native Method)
- parking to wait for <0x000000070fe08998> (a java.util.concurrent.locks.ReentrantLock\$Sync)
at java.util.concurrent.locks.LockSupport.park(java.base@21.0.2/LockSupport.java:403)
at java.util.concurrent.locks.AbstractQueuedSynchronizer.acquire(java.base@21.0.2/AbstractQueuedSynchronizer.java:1048)
at java.util.concurrent.locks.AbstractQueuedSynchronizer.acquire(java.base@21.0.2/AbstractQueuedSynchronizer.java:1048)
at java.util.concurrent.locks.ReentrantLock\$Sync.lock(java.base@21.0.2/ReentrantLock\$Sync.java:45)
at java.util.concurrent.locks.ReentrantLock.lock(java.base@21.0.2/ReentrantLock.java:254)
at jdk.internal.misc.InternalLock.lock(java.base@21.0.2/InternalLock.java:45)
at java.io.PrintStream.format(java.base@21.0.2/PrintStream.java:1344)
at Hello\$1.run(Hello.java:27)
- locked <0x000000070fe120e8> (a Deadlockable)
at java.lang.Thread.runWith(java.base@21.0.2/Thread.java:1596)
at java.lang.Thread.run(java.base@21.0.2/Thread.java:1583)

Found 1 deadlock.

This is obviously bad code because it unnecessarily calls `std::print` / `System.out.format` under a lock but it is still undesirable to have it deadlocked.

To prevent deadlocks while still providing major performance improvements and preventing dynamic allocations for the common case, this paper proposes making user-defined formatters opt into the new behavior. Standard formatters are nonlocking and will be opted in which means that `std::print` can

be used as a replacement for all current uses of `printf` without concerns that it causes unbounded memory allocation.

The opt in is done via the variable template similarly to `enable_borrowed_range` and `format_kind`:

```
struct foo {};  
  
template <> struct std::formatter<foo> {  
    // ...  
};  
  
template <> constexpr bool std::has_locking_formatter<foo> = false;
```

R2 of the paper used the `static constexpr bool` member variable:

```
template <> struct std::formatter<foo> {  
    static constexpr bool locking = false;  
    // ...  
};
```

The advantage of the variable template is that it doesn't propagate when one `formatter` is inherited from another.

§ 7. Performance

The following benchmark demonstrates the difference in performance between different implementation strategies using the reference implementation of `print` from [FMT]. This benchmark is based on the one from [P2093] but modified to avoid the small string optimization effects. It formats a simple message and prints it to the output stream redirected to `/dev/null`. It uses the Google Benchmark library [GOOGLE-BENCH] to measure timings:

```
#include <cstdio>  
#include <benchmark/benchmark.h>  
#include <fmt/format.h>  
  
void printf(benchmark::State& s) {  
    while (s.KeepRunning())  
        std::printf("The answer to life, the universe, and everything is %d.\n",
```

```

}
BENCHMARK(printf);

void vprint_string(fmt::string_view fmt, fmt::format_args args) {
    auto s = fmt::vformat(fmt, args);
    int result = fwrite(s.data(), 1, s.size(), stdout);
    if (result < s.size()) throw fmt::format_error("fwrite error");
}

template <typename... T>
void print_string(fmt::format_string<T...> fmt, T&&... args) {
    vprint_string(fmt, fmt::make_format_args(args...));
}

void print_string(benchmark::State& s) {
    while (s.KeepRunning()) {
        print_string("The answer to life, the universe, and everything is {}.\\n",
    }
}
BENCHMARK(print_string);

void vprint_stack(fmt::string_view fmt, fmt::format_args args) {
    auto buf = fmt::memory_buffer();
    fmt::vformat_to(std::back_inserter(buf), fmt, args);
    int result = fwrite(buf.data(), 1, buf.size(), stdout);
    if (result < buf.size()) throw fmt::format_error("fwrite error");
}

template <typename... T>
void print_stack(fmt::format_string<T...> fmt, T&&... args) {
    vprint_stack(fmt, fmt::make_format_args(args...));
}

void print_stack(benchmark::State& s) {
    while (s.KeepRunning()) {
        print_stack("The answer to life, the universe, and everything is {}.\\n",
    }
}
BENCHMARK(print_stack);

void print_direct(benchmark::State& s) {
    while (s.KeepRunning())
        fmt::print("The answer to life, the universe, and everything is {}.\\n",

```

```

}
BENCHMARK(print_direct);

BENCHMARK_MAIN();

```

Here `print_string` formats into a temporary string, `print_stack` formats into a buffer allocated on stack and `print_direct` formats directly into the C stream buffer under a lock. `printf` is included for comparison.

The benchmark was compiled with Apple clang version 15.0.0 (clang-1500.1.0.2.5) with `-O3 -DNDEBUG` and run on macOS 14.2.1 with M1 Pro CPU. Below are the results:

```

Run on (8 X 24 MHz CPU s)
CPU Caches:
  L1 Data 64 KiB
  L1 Instruction 128 KiB
  L2 Unified 4096 KiB (x8)
Load Average: 5.03, 3.99, 3.89

```

Benchmark	Time	CPU	Iterations
<code>printf</code>	81.8 ns	81.5 ns	8496899
<code>print_string</code>	88.5 ns	88.2 ns	7993240
<code>print_stack</code>	63.8 ns	61.9 ns	11524151
<code>print_direct</code>	51.3 ns	51.0 ns	13846580

Note that estimated CPU frequency is incorrect.

On Linux (Ubuntu 22.04.3 LTS) with gcc 11.4.0, glibc/libstdc++ and Intel Core i9-9900K CPU the results are similar except that `printf` is slightly faster than `print` with the stack-allocated buffer optimization:

```

Run on (16 X 3600 MHz CPU s)
CPU Caches:
  L1 Data 32 KiB (x8)
  L1 Instruction 32 KiB (x8)
  L2 Unified 256 KiB (x8)
  L3 Unified 16384 KiB (x1)
Load Average: 0.00, 0.00, 0.00

```

Benchmark	Time	CPU	Iterations
<code>printf</code>	52.1 ns	52.1 ns	13386398
<code>print_string</code>	65.7 ns	65.7 ns	10674838
<code>print_stack</code>	55.8 ns	55.8 ns	12535414
<code>print_direct</code>	46.3 ns	46.3 ns	15087266

Direct output is 42-72% faster than writing to a temporary string and 21-24% faster than writing to a stack-allocated buffer on this benchmark.

Preliminary testing in `libstc++` showed ~25% improvement compared to the existing implementation.

§ 8. Implementation

This proposal has been implemented in the open-source `{fmt}` library ([\[FMT\]](#)) bringing major performance improvements.

§ 9. Wording

Update the value of the feature-testing macro `__cpp_lib_print` to the date of adoption in [\[version.syn\]](#).

Modify [\[format.syn\]](#) as indicated:

```
// [format.formatter], formatter
template<class T, class charT = char> struct formatter;

template<class T>
    constexpr bool has_locking_formatter = true;

...

```

Add a new clause `[format.formatter.locking]` to [\[format.formatter\]](#):

```
template<class T>
    constexpr bool has_locking_formatter = true;

```

Remarks: Pursuant to `[namespace.std]`, users may specialize `has_locking_formatter` for cv-unqualified program-defined types. Such specializations shall be usable in constant expressions (`[expr.const]`) and have type `const bool`.

Modify `[print.fun]` as indicated:

...

```
template<class... Args>
    void print(FILE* stream, format_string<Args...> fmt, Args&&... args);
```

Effects:

Let `locksafe` be `!(has_locking_formatter<remove_cvref_t<Arg>> || ...)`.

If the ordinary literal encoding (`[lex.charset]`) is UTF-8, equivalent to:

```
vprint_unicode(stream, fmt.str, make_format_args(args...));

locksafe ?
    vprint_unicode_locking(stream, fmt.str, make_format_args(args...)) :
    vprint_unicode(stream, fmt.str, make_format_args(args...));
```

Otherwise, equivalent to:

```
vprint_nonunicode(stream, fmt.str, make_format_args(args...));

locksafe ?
    vprint_nonunicode_locking(stream, fmt.str, make_format_args(args...)) :
    vprint_nonunicode(stream, fmt.str, make_format_args(args...));
```

...

```
template<class... Args>
    void println(FILE* stream, format_string<Args...> fmt, Args&&... args);
```

Effects: Equivalent to:

```
print(stream, "{}\n", format(fmt, std::forward<Args>(args)...));  
print(stream, runtime format(string(fmt.get()) + '\n'), std::forward<Args>
```

```
void vprint_unicode(FILE* stream, string_view fmt, format_args args);
```

Effects: Equivalent to:

```
vprint_unicode_locking(stream, "{}", vformat(fmt, args));
```

```
void vprint_unicode_locking(FILE* stream, string_view fmt, format_args args);
```

Preconditions: `stream` is a valid pointer to an output C stream.

Effects: ~~The function initializes an automatic variable via~~

```
string out = vformat(fmt, args);
```

Let `out` denote the character representation of formatting arguments provided by `args` formatted according to specifications given in `fmt`.

Locks `stream`. If `stream` refers to a terminal capable of displaying Unicode, writes `out` to the terminal using the native Unicode API; if `out` contains invalid code units, the behavior is undefined and implementations are encouraged to diagnose it. Otherwise writes `out` to `stream` unchanged. If the native Unicode API is used, the function flushes `stream` before writing `out`. Releases the lock.

...

```
void vprint_nonunicode(FILE* stream, string_view fmt, format_args args);
```

Effects: Equivalent to:

```
vprint_nonunicode_locking("{ }", vformat(fmt, args)).;
```

```
void vprint_nonunicode_locking(FILE* stream, string_view fmt, format args);
```

Preconditions: `stream` is a valid pointer to an output C stream.

Effects: Writes the result of `vformat(fmt, args)` to `stream`. Locks `stream`, writes the character representation of formatting arguments provided by `args` formatted according to specifications given in `fmt` to `stream` and releases the lock.

Throws: Any exception thrown by the call to `vformat` ([`format.err.report`]). `system_error` if writing to `stream` fails. May throw `bad_alloc`.

...

Modify [`format.formatter.spec`] as indicated:

...

Let `charT` be either `char` or `wchar_t`. Each specialization of `formatter` is either enabled or disabled, as described below. A *debug-enabled* specialization of `formatter` additionally provides a public, constexpr, non-static member function `set_debug_format()` which modifies the state of the `formatter` to be as if the type of the *std-format-spec* parsed by the last call to `parse` were `?`. Each header that declares the template `formatter` provides the following enabled specializations:

- The debug-enabled specializations

```
template<> struct formatter<char, char>;  
template<> struct formatter<char, wchar_t>;  
template<> struct formatter<wchar_t, wchar_t>;
```

...

The `parse` member functions of these formatters interpret the format specification as a *std-format-spec* as described in [`format.string.std`].

For types that have these formatter `has_locking_formatter` is specialized to `false`.

[Note 1: Specializations such as `formatter<wchar_t, char>` and `formatter<const char*, wchar_t>` that would require implicit multibyte / wide string or character conversion are disabled. — end note]

...

§ 10. Acknowledgements

Thanks to Jonathan Wakely for implementing the proposal in libstdc++, providing benchmark results and suggesting various improvements to the paper.

Thanks to Ben Craig for proposing to make user-defined formatters opt into the new behavior to prevent potential deadlocks.

§ References

§ Informative References

[FMT]

Victor Zverovich; et al. *The {fmt} library*. URL: <https://github.com/fmtlib/fmt>

[GOOGLE-BENCH]

Google Benchmark: A microbenchmark support library. URL: <https://github.com/google/benchmark>

[LWG4042]

LWG Issue 4042: `std::print` should permit an efficient implementation. URL: <https://cplusplus.github.io/LWG/issue4042>

[N2310-STREAMS]

7.21.2 Streams. ISO/IEC 9899:202x. Programming languages — C. URL: <https://www.open-std.org/jtc1/sc22/wg14/www/docs/n2310.pdf#page=233>

[P2093]

Victor Zverovich. *Formatted output*. URL: <https://wg21.link/p2093>

[STDIO-LOCK]

The Open Group Base Specifications Issue 7, 2018 edition. IEEE Std 1003.1-2017. flockfile, ftrylockfile, funlockfile - stdio locking functions. URL: <https://pubs.opengroup.org/onlinepubs/9699919799/functions/flockfile.html>